

Incident handler's journal

Instructions

As you continue through this course, you may use this template to record your findings after completing an activity or to take notes on what you've learned about a specific tool or concept. You can also use this journal as a way to log the key takeaways about the different cybersecurity tools or concepts you encounter in this course.

Date: 02/09/2026	**Entry:** 1
Description	I reviewed a ransomware attack against a small healthcare clinic in the U.S. The attack started with a phishing email, and the malware encrypted important files. Staff could not access patient records, so the clinic had to stop its normal work.
Tool(s) used	None.
The 5 W's	**Who:** An organized group of hackers. The group was not named in the scenario. **What:** Ransomware encrypted important files and stopped the clinic from working normally. **When:** Tuesday at around 9:00 a.m. The exact date was not given. **Where:** A small healthcare clinic in the United States. **Why:** The attackers got in through a phishing email with a malicious attachment. They wanted money in exchange for a decryption key.
Additional notes	**1.** Was any patient information stolen before the files were encrypted? **2.** Does the clinic have backups it can use to recover its files? **3.** What could the clinic do to help employees recognize phishing emails? **4.** The clinic shut down its systems and contacted other organizations for help.

Date: 05/10/2026	**Entry:** 2
Description	I looked into a suspicious file from an employee's computer at a financial services company. I searched the file's hash on VirusTotal and checked its detections, related domains, and behavior. The results pointed to Flagpro malware.
Tool(s) used	VirusTotal. I checked the Detection, Details, Relations, and Behavior tabs to learn more about the file.
The 5 W's	**Who:** An unknown attacker. Flagpro has been linked to BlackTech, but this does not prove who was behind this case. **What:** An employee opened a password-protected spreadsheet sent by email. Malicious code ran and created unauthorized executable files. **When:** Between around 1:11 p.m. and 1:20 p.m. The date was not provided. **Where:** An employee's computer at a financial services company. **Why:** The file was opened and its malicious code ran. The scenario did not explain the attacker’s final goal.
Additional notes	**1.** VirusTotal showed 53 of 71 security vendors detecting the file, with a community score of -295. **2.** One of the suspicious domains linked to the file was org[.]misecure[.]com. **3.** The Behavior tab showed activity related to credentials and command and control. **4.** I learned that it helps to check more than just the hash when investigating a file.

Date: 05/10/2026	**Entry:** 3
Description	I reviewed a phishing email sent to the HR team at Inergy. I used the phishing playbook and the results of my previous VirusTotal check to decide what to do with the alert. The evidence supported escalating the ticket to a level-two analyst.
Tool(s) used	Phishing response playbook and VirusTotal. I used the playbook to check when an alert should be escalated and the previous hash results to check the attachment.
The 5 W's	**Who:** An unknown attacker using the sender name “Def Communications.” **What:** A phishing email pretending to be a job application included a malicious attachment called bfsvc.exe. **When:** Wednesday, July 20, 2022, at 9:30:14 a.m. **Where:** The HR inbox at Inergy (hr@inergy.com). **Why:** The attacker tried to get HR to open an infected file by disguising it as a resume.
Additional notes	**1.** Ticket A-2703 had Medium severity and was escalated. **2.** The file hash matched the one I had already checked in VirusTotal. **3.** The sender details did not match, the email had writing mistakes, and the attachment was an executable file. **4.** Based on the playbook and the malicious file, I escalated the alert to SOC level two.

Date: 14/10/2026	**Entry:** 4
Description	I used Wireshark to look at network traffic from a sample PCAP file. I filtered packets by IP address, MAC address, and port. I also looked at DNS requests and HTTP traffic to understand how the devices were communicating.
Tool(s) used	Wireshark. 1. Opened the sample PCAP file to look through the captured packets. 2. Used display filters to focus on specific IP addresses, MAC addresses, and ports. 3. Checked DNS responses and details of TCP packets. 4. Looked for the text “curl” in TCP traffic.
The 5 W's	**Who:** N/A **What:** N/A **When:** N/A **Where:** N/A **Why:** N/A
Additional notes	**1.** I used filters including ip.addr, ip.src, ip.dst, eth.addr, udp.port, and tcp.port. **2.** The DNS response for opensource.google.com included the IP address 142.250.1.139. **3.** I looked at traffic on TCP port 80 and checked information such as TTL and TCP flags. **4.** The filter tcp contains "curl" helped find packets containing that text.

Date: 14/10/2026	**Entry:** 5
Description	I used tcpdump on a Linux virtual machine to capture network traffic. I checked which interfaces were available, captured packets from eth0, and saved web traffic to a PCAP file. I then reopened the capture and checked the packet details.
Tool(s) used	tcpdump, ifconfig, and curl. 1. Used ifconfig and tcpdump -D to check the available interfaces. 2. Captured packets from eth0 using tcpdump. 3. Captured HTTP traffic on port 80 and saved it as capture.pcap. 4. Used curl to create traffic and tcpdump to look at the saved file.
The 5 W's	**Who:** N/A **What:** N/A **When:** N/A **Where:** N/A **Why:** N/A
Additional notes	**1.** The -i option chooses the interface, -c limits the number of packets, and -w saves the capture. **2.** I used -nn to avoid resolving names and -r to read the saved PCAP file. **3.** The -v option showed more packet details, while -X showed hexadecimal and ASCII data. **4.** It was useful to see that I could capture traffic and check it again later.

Date: 14/10/2026	**Entry:** 6
Description	I used Suricata to test a rule that detects HTTP GET requests. I ran the rule against a sample PCAP file and looked at the alerts that were generated. I also used jq to read the JSON logs and compare information from different events.
Tool(s) used	Suricata and jq. 1. Read the rule in custom.rules to see which HTTP requests it would match. 2. Ran Suricata with sample.pcap and custom.rules. 3. Checked fast.log to see the alerts that were created. 4. Used jq to find specific information in eve.json, such as timestamps and flow IDs.
The 5 W's	**Who:** N/A **What:** N/A **When:** N/A **Where:** N/A **Why:** N/A
Additional notes	**1.** The rule matched HTTP GET requests and created alerts saying “GET on wire.” **2.** The rule used sid 12345 and rev 3. The test alerts had priority 3. **3.** fast.log was easier to read quickly, while eve.json contained more details. **4.** I used flow_id to find events that belonged to the same network flow. **5.** A normal GET request can trigger this rule. That does not mean it is an attack.

Reflections/Notes

1. Were there any specific activities that were challenging for you? Why or why not?

The tcpdump and Suricata labs were a bit harder for me. I could follow the commands, but I wanted to understand what options like -i, -r, and -S actually did instead of just copying them. After going back over the commands and their outputs, they made more sense.

2. Has your understanding of incident detection and response changed since taking this course?

Before this course, I knew that security teams checked alerts, but I didn't really know what happened next. Now I have a better idea of how analysts use logs, check evidence, document what they find, and decide when to escalate an incident. I still need more hands-on practice, but I understand the process much better.

3. Was there a specific tool or concept that you enjoyed the most? Why?

Suricata was probably the most interesting tool for me. It was useful to see how a rule can match traffic and create an alert, and then to check what happened in fast.log and eve.json. I also liked using jq to look for specific information instead of reading the whole log.